॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
Indian Institute of Technology Dharwad

CS101C: Introduction to Programming (Using C)

Instructors: Prof. Aikta Arya and Prof. Konjengbam Anand

CSE Department

Indian Institute of Technology, Dharwad

Week 3

Today's Class (17/8/2026)

Arithmetic Expressions

- Suppose x and y are variables
- $x + y, x - y, x * y, x/y, x\%y$: binary arithmetic
- A simple statement: $y = x + 3 * x/(y - 4);$
- Numeric literals like 3 or 4 valid in expressions.
- Semicolon ends statement (not newline)
- $x+ = y, x- = y, x* = y, x/ = y, x\% = y$: arithmetic and assignment.

Order of Operations

Operator	Evaluation Direction
+, − (sign)	Right-to-left
*, /, %	Left-to-right
+, −	Left-to-right
=, +=, −=, *=, /=, %=	Right-to-left

Remember

Use parentheses () to override the order of evaluation.

Arithmetic Expressions

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
In Indian Institute of Technology Dharwad

Example:

$$x = 2.0, \qquad y = 6.0$$

Evaluate the statement:

```
float z = x + 3*x/(y-4);
```

Arithmetic Expressions

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
In Indian Institute of Technology Dharwad

Example:

$$x = 2.0, \qquad y = 6.0$$

Evaluate the statement:

```
float z = x + 3*x/(y-4);
```

Step 1: Evaluate the expression in parentheses

```
float z = x + 3*x/(y-4);
```

```
float z = x + 3*x/2.0;
```

Step 2: Evaluate multiplication and division (left-to-right)

```
float z = x + 6.0/2.0;
```

```
float z = x + 3.0;
```

Arithmetic Expressions

Step 3: Evaluate addition

```
float z = x + 3.0;
```

```
float z = 5.0;
```

Step 4: Perform initialization with assignment

$$\boxed{z = 5.0}$$

Arithmetic Expressions

Step 3: Evaluate addition

```
float z = x + 3.0;
```

```
float z = 5.0;
```

Step 4: Perform initialization with assignment

$$\boxed{z = 5.0}$$

Question: How do we insert parentheses to get $z = 4.0$?

```
float z = (x + 3*x)/(y - 4);
```

Precedence	Operator	Description	Associativity
1	++ --	Suffix/postfix increment and decrement	Left-to-right
	()	Function call	
	[]	Array subscripting	
	.	Structure and union member access	
	->	Structure and union member access through pointer	
	(*type*){*list*}	Compound literal(C99)	
2	++ --	Prefix increment and decrement[note 1]	Right-to-left
	+ -	Unary plus and minus	
	! ~	Logical NOT and bitwise NOT	
	(*type*)	Cast	
	*	Indirection (dereference)	
	&	Address-of	
	sizeof	Size-of[note 2]	
	_Alignof	Alignment requirement(C11)	
3	* / %	Multiplication, division, and remainder	Left-to-right
4	+ -	Addition and subtraction	
5	<< >>	Bitwise left shift and right shift	
6	< <=	For relational operators < and ≤ respectively	
	> >=	For relational operators > and ≥ respectively	
7	== !=	For relational = and ≠ respectively	
8	&	Bitwise AND	
9	^	Bitwise XOR (exclusive or)	
10	|	Bitwise OR (inclusive or)	
11	&&	Logical AND	
12	||	Logical OR	
13	?:	Ternary conditional[note 3]	Right-to-left
14[note 4]	=	Simple assignment	
	+= -=	Assignment by sum and difference	
	*= /= %=	Assignment by product, quotient, and remainder	
	<<= >>=	Assignment by bitwise left shift and right shift	
	&= ^= |=	Assignment by bitwise AND, XOR, and OR	
15	,	Comma	Left-to-right

source:
https://en.cppreference.com/w/c/language/operator_precedence.html

```
a=10, b=20, c=5. Evaluate:

d=a+b*c

d=a*b/c

d=b<a*c

d+=a=b

d+=a?b:c;

d=a+-b
```

challenge Q: `d=++a+-b;?`

Order of Operations – Practice

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
In Indian Institute of Technology Dharwad

$$a = 10, \qquad b = 20, \qquad c = 5$$

1.

$$\texttt{d = a + b*c}$$

$$= 10 + (20)(5) = 10 + 100 = \boxed{110}$$

2.

$$\texttt{d = a*b/c}$$

$$= (10)(20)/5 = 200/5 = \boxed{40}$$

3.

$$\texttt{d = b < a*c}$$

$$20 < (10)(5) \quad \Rightarrow \quad 20 < 50 \quad \Rightarrow \quad \boxed{d = 1}$$

4.

$$\texttt{d += a = b}$$

$$\texttt{a=b} \Rightarrow a = 20$$

$$\texttt{d += 20=20}$$

Order of Operations – Practice

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
In Indian Institute of Technology Dharwad

5.

```
d += a ? b : c
```

Since $a = 10 \neq 0$:

$$\texttt{a ? b : c} \Rightarrow b = 20$$

```
d += 20
```

6.

```
d = a + -b
```

Here, `-b` is a unary minus:

$$-b = -20$$

$$d = 10 + (-20)$$

$$\boxed{d = -10}$$

Order of Operations – Practice

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
In Indian Institute of Technology Dharwad

Challenge:

```
d = ++a + -b
```

$$++a \Rightarrow a = 11, \qquad -b = -20$$

$$d = 11 + (-20)$$

$$\boxed{d = -9}$$

Scanf - a way to accept user input

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
In Indian Institute of Technology Dharwad

```c
#include <stdio.h>
int main()
{
int a;
scanf("%d",&a);
printf("%d",a);
return 0;
}
```

- Reads user input from terminal (stdin) and stores in variables
- & - address of operator.
- Arguments of scanf: 1st = string, 2nd and subsequent (if present) = address
- scanf() returns the number of input items that were successfully matched and assigned. of variables.

1. Variable and Memory

```
int a;
```

Memory (Simplified)

Address	Contents	
1000	?	← This cell belongs to 'a'
1004		
1008		
⋮	⋮	
2000		

What do 'a' and '&a' mean?

- `a` → value stored in 'a'
- `&a` → address of 'a' (which is 1000)



scanf needs the address because it has to put the user's input somewhere in memory.

2. How scanf("%d", &a) works



User enters 25

↓

scanf( ) is called

↓

"%d" → read integer

↓

"&a" → where to store?

↓

Address	Contents
1000	25

← Value 25 is stored at address 1000 (i.e., in variable 'a')

↓

a = 25

Summary:
scanf takes the address (&a) so that it knows exactly where in memory the input should be stored.

Scanf - a way to accept user input




```c
#include <stdio.h>
int main()
{
    int a, b;

    int n = scanf("%d %d", &a, &b);

    printf("%d\n", n);

    return 0;

}
```

Here:
%d → successfully reads 10
%d → successfully reads 20
Therefore: scanf() returns 2
Output:2

Today's Class (19/8/2026)

Control flow

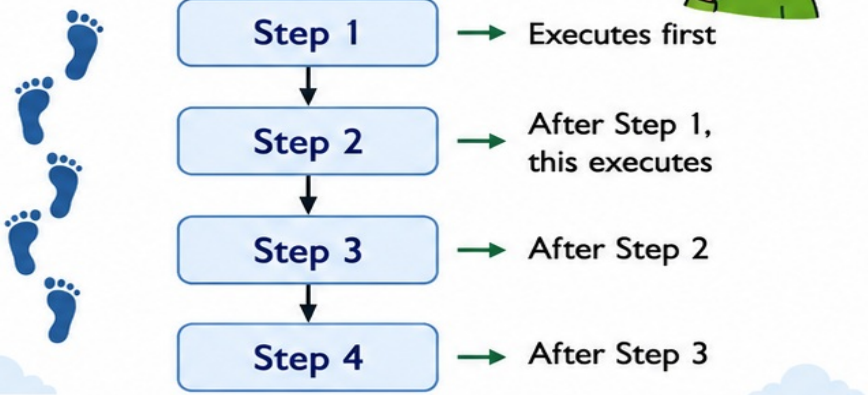
॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड

1 Control Flow: The Straight Road

So far, our program executes one instruction after the previous one — in order.

```
1  printf("Step 1\n");
2  printf("Step 2\n");
3  printf("Step 3\n");
4  printf("Step 4\n");
```



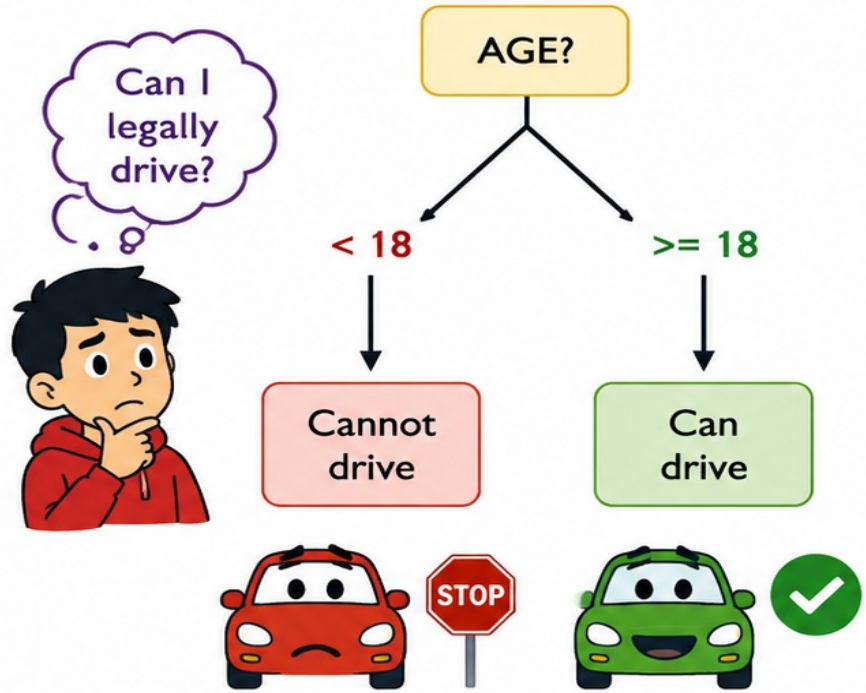




So far, our program has been like a **STRAIGHT ROAD**.

2 Real-Life Decision

Suppose you are 17 years old.
Can you legally drive?



Now our program cannot simply go to the next line.
It has to make a **DECISION**.

Control flow with if-else

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
In Indian Institute of Technology Dharwad

- So far you have seen flow of control, where an instruction on a line executed after the instruction on the previous line.
- The if-else construct in C changes that i.e. the previously executed instruction need not necessarily be the one on the previous line.

```
1  #include <stdio.h>
2  int main()
3  {
4      scanf("%d",&age);
5      if(age<18){
6          printf("cannot drive");
7      } else {
8            printf("drive safe");
9            }
10     printf("bye");
11     return 0;
12 }
```

Here: Line 8 not preceded by 6 & Line 10 not necessarily preceded by 8.

Control flow: Only if

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड

3. First if — no else

Smallest possible example:

```
int age;
scanf("%d", &age);
if (age >= 18)
{
    printf("You can drive\n");
}
printf("Bye\n");
```

Only when the condition is **TRUE** does this block run!

Input: 20



1. Read age → 20
2. Check: age >= 18 ?
   20 >= 18 → **TRUE**
3. Execute: printf("You can drive\n");
4. Execute: printf("Bye\n");

Output:

```
You can drive
Bye
```



Input: 15



1. Read age → 15
2. Check: age >= 18 ?
   15 >= 18 → **FALSE**
3. Skip the if-block
   (not executed)
4. Execute: printf("Bye\n");

Output:

```
Bye
```





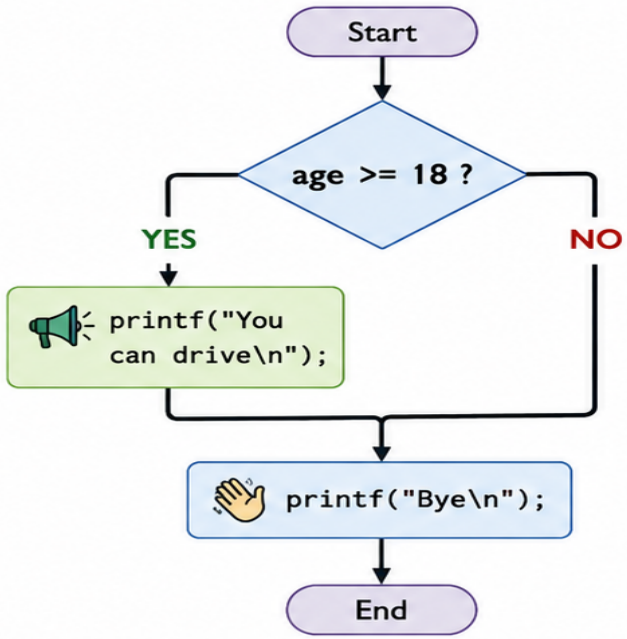
Where did "You can drive" go?
It wasn't executed!



The next line in the code is **not necessarily** the next line that gets executed!

4. Control Flow Visualized

```
if (age >= 18)
{
    printf("You can drive\n");
}
printf("Bye\n");
```





Key Takeaway

With **if**, we create a **DECISION POINT**. Based on the condition, the program chooses **one path** to follow.



Control flow: if-else both

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड

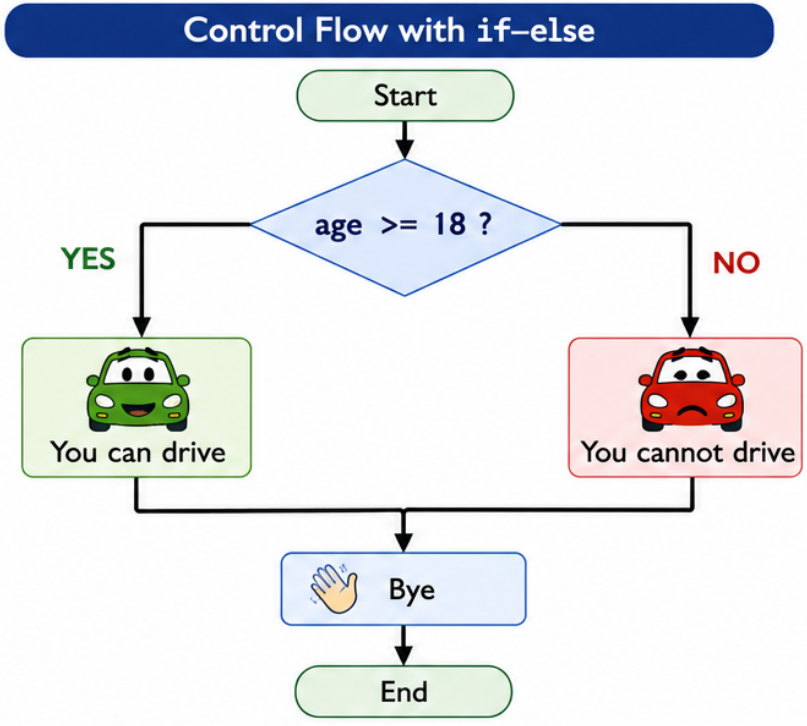
5 Now introduce else



But what if I want to do something when the condition is **false**?

```
if (age >= 18)
{
    printf("You can drive\n");
}
else
{
    printf("You cannot drive\n");
}
printf("Bye\n");
```

Exactly **one** of the **two** blocks executes.



Input: 20

Let age = 20



1. Check: age >= 18 ?
   20 >= 18 → **TRUE**
2. Execute: printf("You can drive\n");
3. Skip the else-block
4. Execute: printf("Bye\n");

Output:

You can drive
Bye



Input: 15

Let age = 15



1. Check: age >= 18 ?
   15 >= 18 → **FALSE**
2. Skip the if-block
3. Execute: printf("You cannot drive\n");
4. Execute: printf("Bye\n");

Output:

You cannot drive
Bye





One condition, two possibilities — but **only one path** is taken!



Control flow with if-else

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
In Indian Institute of Technology Dharwad

Syntax:

```
if (condition) {
    statement1;
    ...
    statementn;
}
else {
    statement1;
    ...
    statementn;
}
```

Important points:

- Curly braces {} can be omitted if there is **only one statement** inside the `if` or `else` block.
- The `else` part is **optional**.
- But if `else` is present, it must be associated with an `if`.

Meaning of `if-else`:

If the condition is **true**, execute the `if`-block.
Otherwise, execute the `else`-block.

Control flow: The Backend Story

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
In Indian Institute of Technology Dharwad

Where does the condition come from?

```
if (age >= 18)
    ...
    // if-block
else
    ...
    // else-block
```

→ **Relational Expression**: `age >= 18` (uses relational operator)

→ **Boolean Result**: TRUE → 1, FALSE → 0 (Always 0 or 1)

→ **Used by if()**: `if ( condition )` → Control Flow (Decides which block executes)

Examples

Example 1: age = 20



→ Evaluate Relational Expression: `20 >= 18`

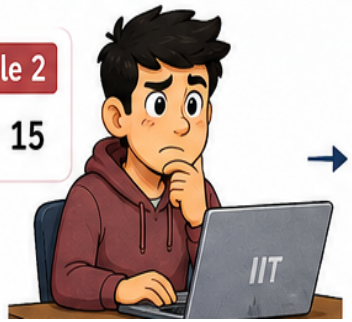
→ Result: TRUE

→ As Integer: 1

→ Control Flow: execute `if-block` { ... }



Example 2: age = 15



→ Evaluate Relational Expression: `15 >= 18`

→ Result: FALSE

→ As Integer: 0

→ Control Flow: execute `else-block` { ... }



Control flow with nested if-else

Syntax:

```
if (condition) {
    if(a>5){x=10;}
    else{x=100;}
}
else {
    if(a>5){x=20;}
    else{x=200;}
}
```

Important points:

- Can have if-else within another if and/or else
- New lines can be skipped. But having new lines makes your code readable.

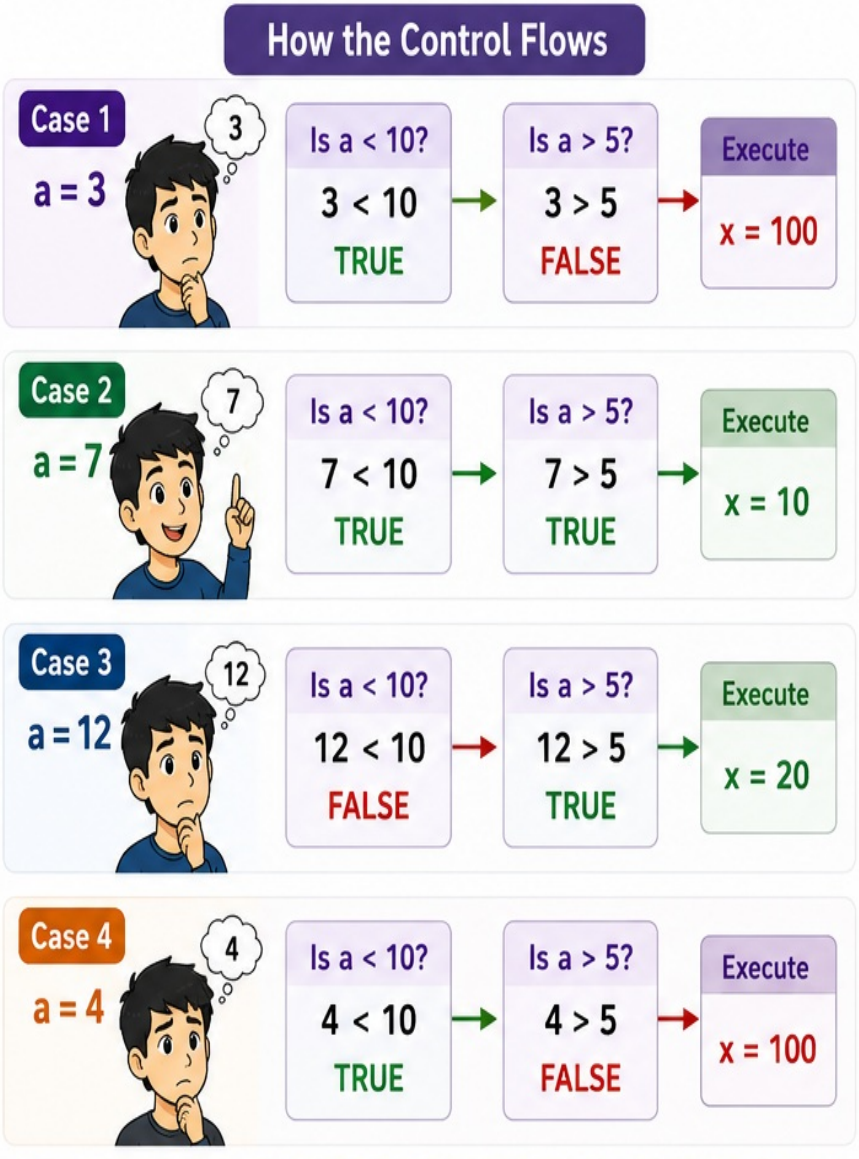
Control flow: The Backend Story

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड

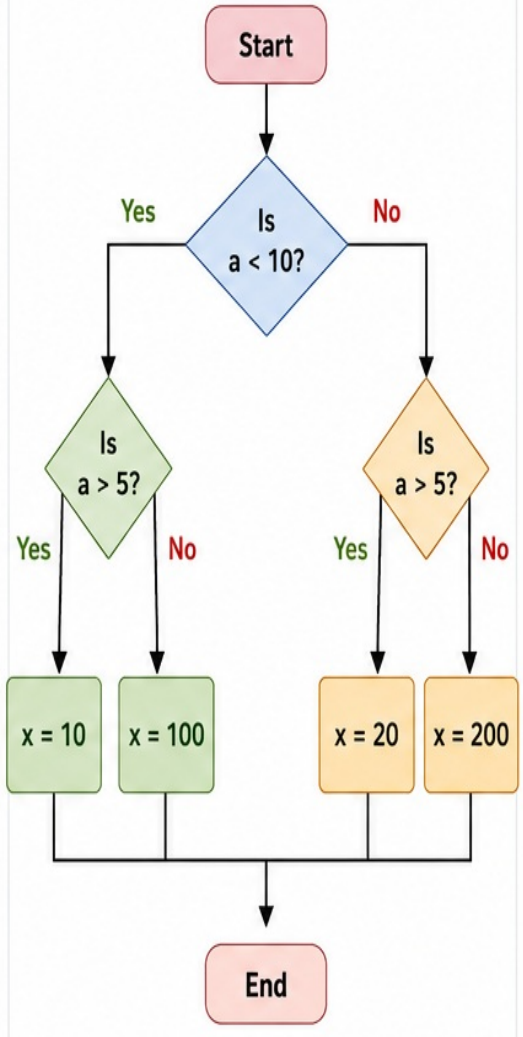
Code

```
if(a<10){
    if(a>5){x=10;}
    else{x=100;}
}
else {
    if(a>5){x=20;}
    else{x=200;}
}
```

How the Control Flows



Control Flow Diagram



Control flow with if-else-if

Syntax:

```
if(a < 10){
    x=10;
} else if (a < 20){
    x=100;
} else if (a<30){
    x=1000;
} else {
    x=10000;
}
```

Important points:

- There is else-if as well:
- The last else part is optional

Control flow: The Backend Story

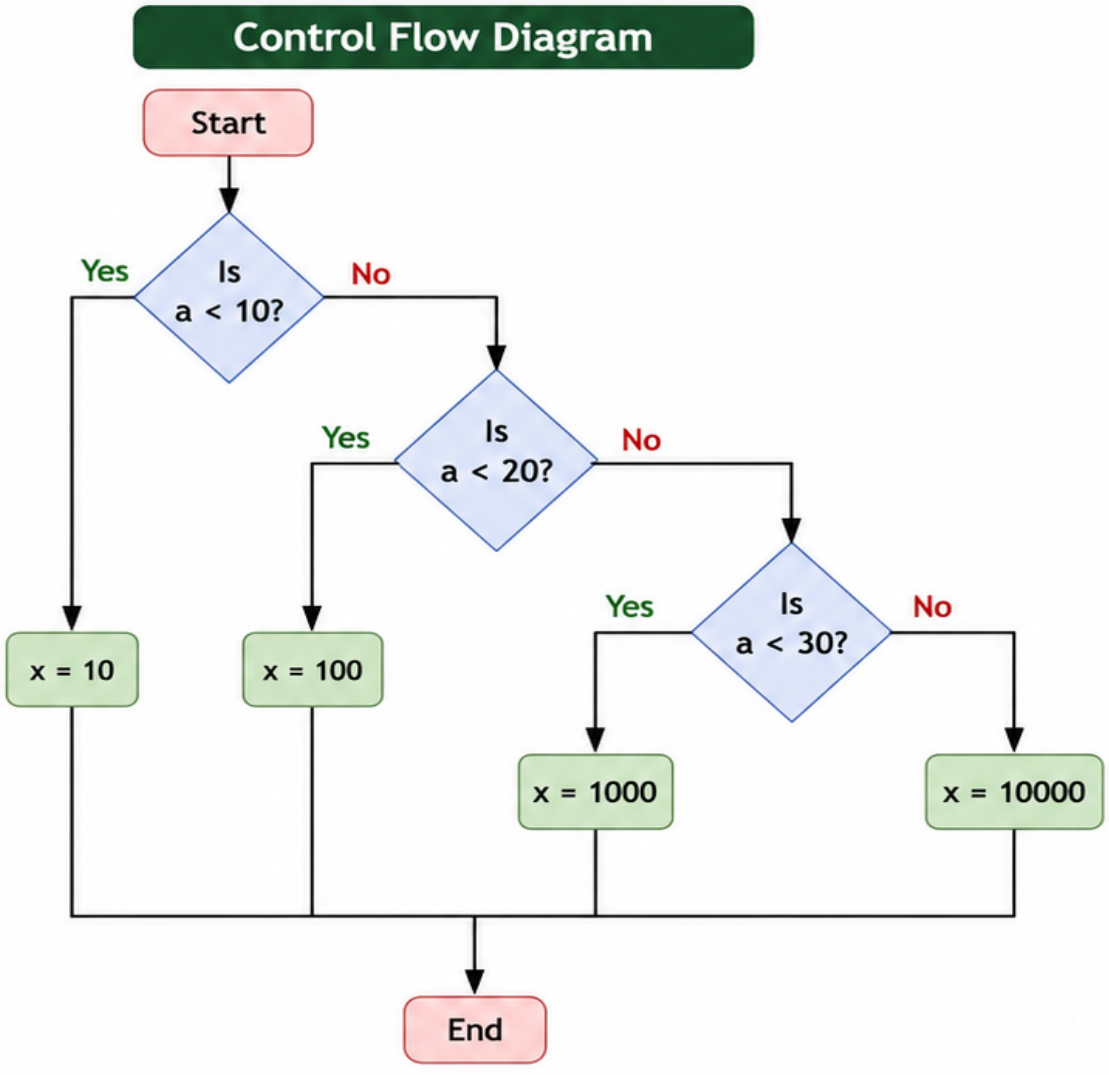
॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
In Indian Institute of Technology Dharwad

Code

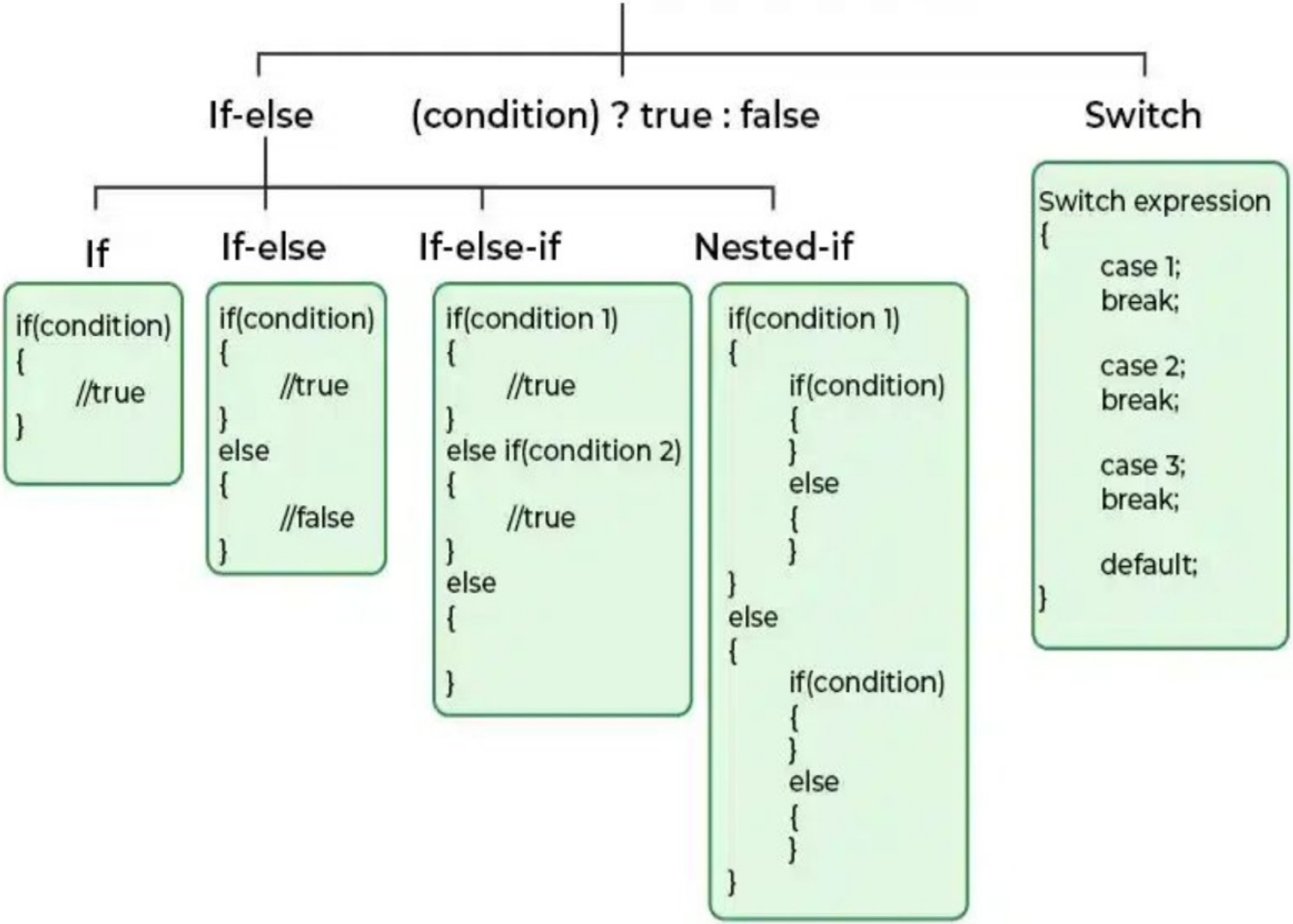
```
if(a < 10){
    x = 10;
} else if (a < 20){
    x = 100;
} else if (a < 30){
    x = 1000;
} else {
    x = 10000;
}
```

How it Works (Examples)

Case	Value of a	Condition Check (in order)	x gets
1	5	5 < 10 → TRUE	x = 10
2	15	15 < 10 → FALSE 15 < 20 → TRUE	x = 100
3	25	25 < 10 → FALSE 25 < 20 → FALSE 25 < 30 → TRUE	x = 1000
4	35	35 < 10 → FALSE 35 < 20 → FALSE 35 < 30 → FALSE else part executes	x = 10000



Conditional Statements in C



Today's Class (21/8/2026)

The Dangling `else` Problem



The Problem

When we have nested `if` statements without braces, it may be confusing to see which `if` an `else` belongs to.

Example

```
if (n > 0)
    if (n > 10)
        printf("n is greater than 10");
    else
        printf("n is not greater than 10");
```

Which `if` does the `else` belong to?

The Dangling `else` Problem



The Problem

When we have nested `if` statements without braces, it may be confusing to see which `if` an `else` belongs to.

Example

```
if (n > 0)
    if (n > 10)
        printf("n is greater than 10");
    else
        printf("n is not greater than 10");
```

Which `if` does the `else` belong to?

C's Rule

The `else` is always attached to the

nearest unmatched `if`.

The Dangling `else` Problem

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड

Recap: if-else

Code

```
if(a < 10)
  if(a < 5)
      printf("a is less than 5\n");
  else
      printf("a is not less than 5\n");
```

Outer if (condition: a < 10)

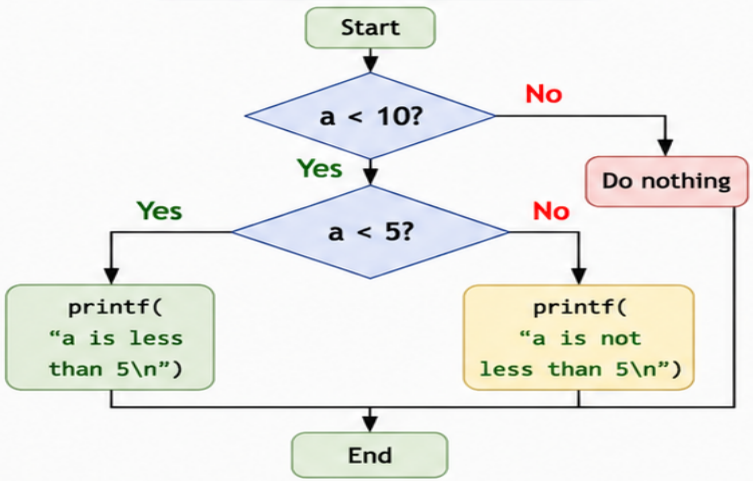
Inner if (condition: a < 5) and its **else**

Which if does else get attached to?

In C, **else** is always attached to the **nearest unmatched if**.

Here, **else** is associated with the **inner if** (a < 5), not the **outer if** (a < 10).

How the Control Flows



Examples

Value of a	a < 10?	a < 5?	Output
3	True	True	a is less than 5
7	True	False	a is not less than 5
12	False	–	*(nothing printed)*
-2	True	True	a is less than 5
10	False	–	*(nothing printed)*

Key Takeaway:
else pairs with the closest **if** that does not yet have an **else**.

Rule: nearest unmatched if

The Dangling `else` Problem

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
In Indian Institute of Technology Dharwad

Consider this code

```
if (n > 0)
    if (n > 10) {
        printf("n is greater than 10");
    }
    else
        printf("n is negative\n");
```

Question

Which `if` does the `else` get attached to?

The Dangling `else` Problem

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
In Indian Institute of Technology Dharwad

Consider this code

```
if (n > 0)
    if (n > 10) {
        printf("n is greater than 10");
    }
    else
        printf("n is negative\n");
```

Question

Which `if` does the `else` get attached to?

The `else` belongs to the nearest unmatched `if`!

The Dangling `else` Problem

So the code is interpreted as

```
if (n > 0) {
    if (n > 10) {
        printf("n is greater than 10");
    }
    else {
        printf("n is negative\n");
    }
}
```

Nested `if-else`: What Gets Printed?

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
In Indian Institute of Technology Dharwad

Consider the following code

```
if (a < 15)
    if (a < 5)
        printf("a is less than 5\n");
    else if (a < 8)
        printf("a is >= 5 but < 8\n");
    else if (a < 12)
        printf("a is >= 8 but < 12\n");
    else
        printf("a is >= 15\n");
```

Question:

What message gets printed when

$$a = 15\ ?$$

Nested `if-else`: What Gets Printed?

When $a = 15$:

$$a < 15$$

is **false**.
Therefore, the outer `if` is not entered.

Result

No message is printed.

Important

The `else` belongs to the **nearest unmatched** `if`. It does **not** belong to the outer `if (a < 15)`.

Control flow with `switch-case`

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
In Indian Institute of Technology Dharwad

- `if-else` gives you **two-way branching**
- with `else-if` you get **multi-way branching**
- `switch` statement in C also gives you **multi-way branching**.

```
switch(class) {

    case 1:
        printf("MON");
        break;

    case 2:
        printf("WED");
        break;

    case 3:
        printf("FRI");
        break;

    default:
        printf("no-class")
}
```

Control flow with `switch-case`



Syntax

```
switch(expression) {

    case constant_expression1: statements;

    case constant_expression2: statements;

    default: statements;
}
```

- `case` is followed by integer-valued constants OR constant expressions.
- `default` is optional.
- For a single set of statements, we can attach multiple `case` constant expressions.
- **Meaning:** If `expression` matches one of the constant expressions, execution begins at the corresponding statements.

Write a C Program



Write a C Program

Write a C program that:

- Accepts a digit from the terminal.
- Uses a `switch` statement to identify the digit.
- Prints the name of the digit that was entered.

Example:

If the user enters:

```
9
```

The output should be:

```
You entered the digit NINE
```

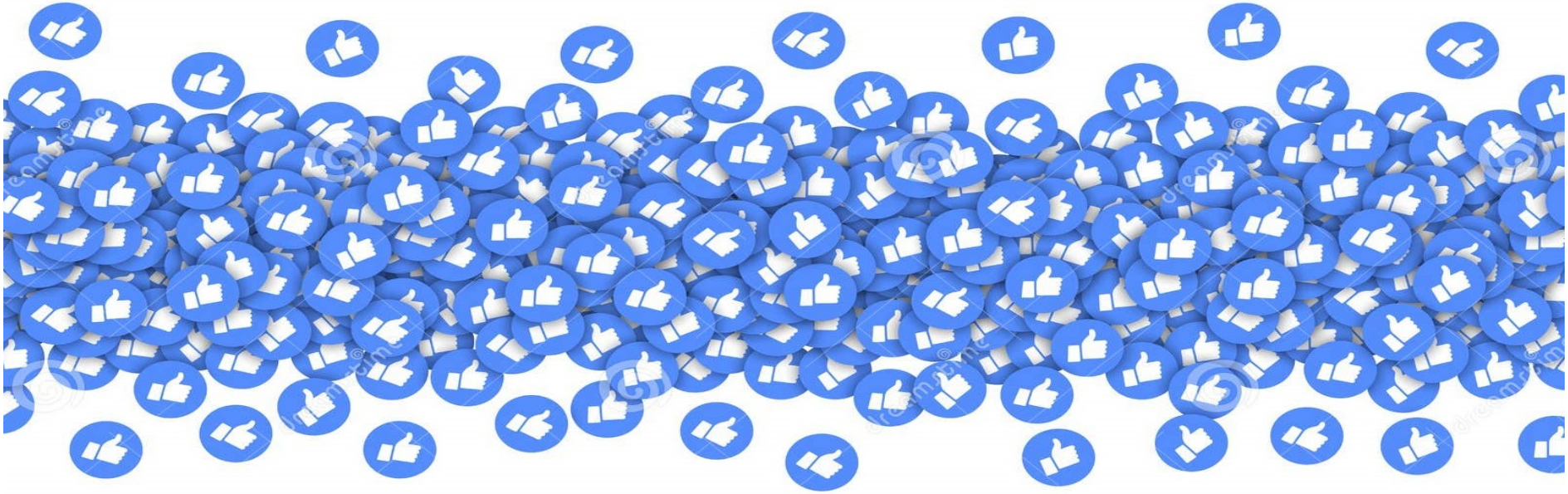
Your Task

Can you write the program using only a `switch` statement?

Write a C Program

॥ सा विद्या या विमुक्तये ॥
भारतीय प्रौद्योगिकी संस्थान धारवाड
In Indian Institute of Technology Dharwad

```
int main() {
    int digit;
    printf("Enter a digit\n");
    scanf("%d", &digit);
    switch(digit) {
        case 0:
        case 1:
        case 2:
            printf("you entered the digit < TWO\n");
        case 3:
            printf("you entered the digit THREE\n");
        case 4:
            printf("you entered the digit FOUR\n");
        case 5:
            printf("you entered the digit FIVE\n");
        case 6:
            printf("you entered the digit SIX\n");
        case 7:
            printf("you entered the digit SEVEN\n");
        case 8:
            printf("you entered the digit EIGHT\n");
        case 9:
            printf("you entered the digit NINE\n");
            break;
        default:
            printf("you entered a non-digit\n");
```



Thank you for your kind attention!!